

Usando tu aplicación como un atacante

10 decisiones de desarrollo que terminan en vulnerabilidades

Luis Diego Raga

Senior Penetration Tester, X-Force Red

10 DECISIONES NORMALES

que un atacante convierte en vulnerabilidades

1

LO QUE PENSAMOS

La suposición

2

LO QUE HACEMOS

El código

3

LO QUE PRUEBA

El atacante

4

LO QUE CAMBIAMOS

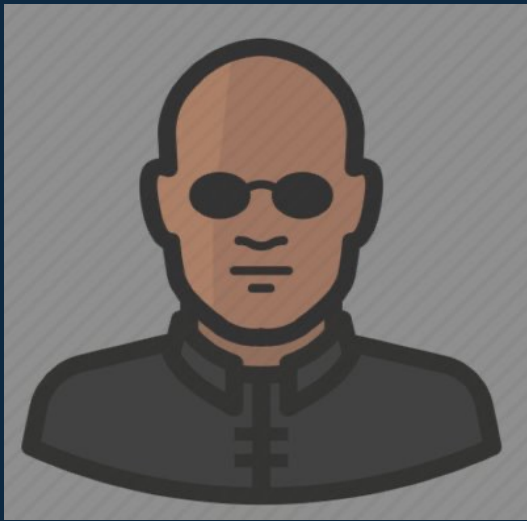
La regla

No vamos a memorizar 50 vulnerabilidades.

Vamos a aprender dos cosas:

- 1. A pensar como un atacante: "¿Cómo podría un atacante abusar de esto?"**
- 2. La segunda la guardamos para el final**

SOBRE MÍ



RAGA

Luis Diego Raga

ROL

Senior Hacker / Penetration Tester

PENTESTING

Web Apps, APIs y AI/LLM

CLOUD SECURITY

DDoS, WAF, Bot Management y CDN

NETWORKING

Firewalls, VPN, IPS e Incident Response

COMUNIDAD

Ethical Hacking Community Lead: DC11506 /
PwnedCR, HTB MVP

LINKEDIN

linkedin.com/in/ldraga

“El usuario no debería cambiar eso”

REQUEST DE LA UI

```
{  
  "productId": 1234,  
  "quantity": 1,  
  "price": 499.99  
}
```

QUÉ CAMBIÓ

La UI no cambió.
El request sí.

REQUEST MODIFICADO

```
{  
  "productId": 1234,  
  "quantity": 1,  
  "price": 1.00  
}
```

IMPACTO

Fraude, cambios de rol, límites evadidos

REGLA

Si viene del cliente, el atacante puede cambiarlo

“Solo voy a concatenar esta variable”

CÓDIGO

```
query = "SELECT * FROM users  
WHERE username = '" + username + "'"
```

INPUT

```
' OR '1'='1
```

El patrón se repite

SQL

Shell

LDAP

Templates

eval()

CÓMO HACERLO MEJOR

Queries parametrizadas y APIs estructuradas. Evitar el shell cuando exista una API equivalente.

REGLA

Los datos deben seguir siendo datos

"Ya está logueado"

AUTHENTICATION

¿Quién eres?

Usuario: Raga

AUTHORIZATION

¿Puede hacer esto?

Ver la factura 9284

REQUEST

GET /api/invoices/9284
Authorization: Bearer eyJ...

REGLA

Cada acción debe validar usuario + acción + recurso

“¿Qué tan difícil puede ser hacer login?”



SHA-256 es rápido. Para passwords necesitamos un algoritmo deliberadamente lento.

“Pongamos el API key aquí por ahora”

FRONTEND

```
const API_KEY =  
  "sk_live_xxxxxxxxx";
```

COMMIT

```
git add .  
git commit -m "initial commit"
```

Si el navegador puede verlo, el usuario puede verlo.

Git tiene una memoria excelente.

CONTROLES Secret manager, credenciales de corta duración, rotación y secret scanning

REGLA

Todo secreto debe poder rotarse cuando se filtre

"Si no se ve, nadie lo va a encontrar"

MENÚ DEL USUARIO

/dashboard
/profile
/orders

LO QUE TAMBIÉN EXISTE

/admin-super-secret-2026
/api/admin/users/delete
/swagger
/debug

URL secreta

+ autorización

URL secreta sin autorización

REGLA

Ocultar una funcionalidad no la protege

“El token es válido, entonces todo está bien”

LOGOUT EN EL FRONTEND

BROWSER

```
localStorage.removeItem("token")
```

TOKEN ROBADO

REQUEST

```
Authorization: Bearer eyJhbGciOi...
```

30 días

de validez

Para la aplicación, quien posee el token sigue siendo el usuario.

REGLA

Tratar cada token como una contraseña temporal

“Ese error me ayuda a debuggear”

RESPUESTA AL USUARIO

Something went wrong.
Reference: ERR-92817

LOG INTERNO

SQLSTATE[HY000]
server=db-prod-02.internal
file=/app/auth/login.py:247

El atacante no necesita conocer:

rutas internas queries versiones hostnames

Los logs tampoco deben guardar passwords, tokens o API keys.

REGLA

Usuario: contexto mínimo. Log interno: detalle útil y seguro

“npm install y seguimos”

TERMINAL

```
npm install super-awesome-package
```

¿Cuánto código acaba de entrar a la aplicación?

Directa

Transitiva

Build tool

Container

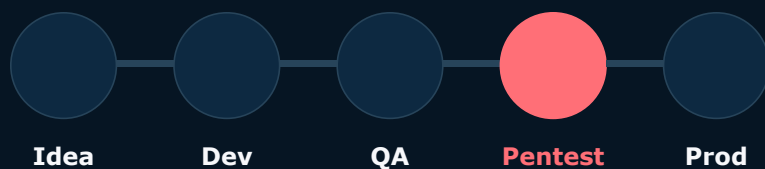
CVE • abandono • typosquatting • dependency confusion

REGLA

Instalar una dependencia también es agregar código

"Security lo revisa antes de producción"

MODELO CARO



MODELO MÁS BARATO

- 1 Requisitos
- 2 Threat model
- 3 Code review
- 4 SAST / SCA
- 5 Pentest

Shift left significa encontrar problemas cuando todavía son baratos de corregir.

REGLA

El pentest valida. No reemplaza el diseño seguro

LAS 10 FRASES QUE DEBERÍAN PREOCUPARNOS

01 El usuario no debería cambiar eso

02 Solo voy a concatenar esta variable

03 Ya está logueado

04 ¿Qué tan difícil puede ser hacer login?

05 Pongamos el API key aquí por ahora

06 Si no se ve, nadie lo va a encontrar

07 El token es válido

08 Ese error me ayuda a debuggear

09 npm install y seguimos

10 Security lo revisa después

Cuando escuchen una de estas frases, hagan una pregunta más.

2. Toda vulnerabilidad comienza con una suposición

Los atacantes prueban esas suposiciones.

PREGUNTAS



Luis Diego Raga

Sr. Penetration Tester, X-Force Red |
Hack the Box MVP | DC11506 Admin ...



[linkedin.com/in/ldraga](https://www.linkedin.com/in/ldraga)